explain详解与索引最佳实践

使用EXPLAIN关键字可以模拟优化器执行SQL语句，分析你的查询语句或是结构的性能瓶颈；

在 select 语句之前增加 explain 关键字，MySQL 会在查询上设置一个标记，执行查询会返回执行计划的信息，而不是执行这条SQL；

注意：如果 from 中包含子查询，仍会执行该子查询，将结果放入临时表中；

我们既然要学习 explain，那我们就要创建一些表；

```
# 演员表
DROP TABLE IF EXISTS `actor`;
CREATE TABLE `actor` (
    `id` int(11) NOT NULL,
    `name` varchar(45) DEFAULT NULL,
    `update_time` datetime DEFAULT NULL,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
INSERT INTO `actor` (`id`, `name`, `update_time`) VALUES (1,'a','2017-12-2 15:27:18'), (2,'b','201

# 电影表
DROP TABLE IF EXISTS `film`;
CREATE TABLE `film` (
    `id` int(11) NOT NULL AUTO_INCREMENT,
    `name` varchar(10) DEFAULT NULL,
    PRIMARY KEY (`id`),
    KEY `idx_name` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
INSERT INTO `film` (`id`, `name`) VALUES (3,'film0'),(1,'film1'),(2,'film2');

# 演员和电影的关联表
DROP TABLE IF EXISTS `film_actor`;
```

```
CREATE TABLE `film_actor` (
    `id` int(11) NOT NULL,
    `film_id` int(11) NOT NULL,
    `actor_id` int(11) NOT NULL,
    `remark` varchar(255) DEFAULT NULL,
    PRIMARY KEY (`id`),
    KEY `idx_film_actor_id` (`film_id`,`actor_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
INSERT INTO `film_actor` (`id`, `film_id`, `actor_id`) VALUES (1,1,1),(2,1,2),(3,2,1);
```

explain 两个变种

（1）**explain extended**：会在 explain 的基础上额外提供一些查询优化的信息。紧随其后通过 **show warnings** 命令可以得到优化后的查询语句，从而看出优化器优化了什么。额外还有 filtered 列，是一个半分比的值，rows * filtered/100 可以估算出将要和 explain 中前一个表进行连接的行数（前一个表指 explain 中的id值比当前表id值小的表）。

```
mysql> explain extended select * from film where id = 1;
mysql> show warnings;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	film	(Null)	const	PRIMARY	PRIMA	4	const	1	100	(Null)

Level	Code	Message
Warning	1681	'EXTENDED' is deprecated and will be removed in a future release.
Note	1003	/* select#1 */ select '1' AS `id`,'film1' AS `name` from `tuling_mysql_db`.`film` where 1 ← 优化的SQL

（2）**explain partitions**：相比 explain 多了个 partitions 字段，如果查询是基于分区表的话，会显示查询将访问的分区。

explain 中的列

```
explain select * from actor;
```

以上的执行计划结果：

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	actor	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	100	(Null)

1. id 列

id 列的编号是 select 的序列号，有几个 select 就有几个id，并且 id 的出现顺序是按 select 出现的顺序增长的。

id 列越大执行优先级越高，id 相同则从上往下执行，id为NULL最后执行。

2. select_type 列

select_type 表示对应行是简单还是复杂的查询。简单查询只有 simple，复杂查询有：primary，subquery，derived，union；

（1）**simple**：简单查询。查询不包含 子查询 和 union；

mysql> explain select * from film where id = 2;

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	film	(Null)	const	PRIMARY	PRIMA	4	const	1	100	(Null)

（2）**primary**：最外层的 select ；----复杂查询
（3）**subquery**：包含在 select 中的子查询（**不在 from 子句中**） -- 复杂查询
（4）**derived**：包含在 from 子句中的子查询。MySQL会将结果存放在一个临时表中，也称为派

生表（derived的英文含义）； --复杂查询
用下面的例子了解 primary、subquery 和 derived 类型：

```
mysql> set session optimizer_switch='derived_merge=off'; #关闭mysql5.7新特性对衍生表的合并优化
mysql> explain select (select 1 from actor where id = 1) from (select * from film where id = 1) de
mysql> set session optimizer_switch='derived_merge=on'; #还原默认配置
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived3>	(Null)	system	(Null)	(Null)	(Null)	(Null)	1	100	(Null)
3	DERIVED	film	(Null)	const	PRIMARY	PRIMARY	4	const	1	100	(Null)
2	SUBQUERY	actor	(Null)	const	PRIMARY	PRIMARY	4	const	1	100	Using index

分析SQL的执行顺序： 先执行 film 表的查询 --> 再执行 actor 表的查询 --> 最后再执行生成的临时表查询；

第1行（ id 为 1）的 select_type 为 primary，表示的最外层的查询；id 为 1，是最小的，则最后执行；

第2行（ id 为 3）的 select_type 为 derived，表示的 from 后面的子查询，也叫派生表，别名为 der 的派生表；id 为 3，是最大的，则最先执行的；

第3行（id 为 2）的 select_type 为 subquery，表示的 from 前面的子查询；id 为 2，是中间的，则在中间执行；

（5）**union**：在 union 中的第二个和之后的 select 都为 union；

```
mysql> explain select 1 union all select 1;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	No tables use
2	UNION	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	No tables use

3. table 列

这一列表示 explain 的一行正在访问哪个表。

当 from 子句中有子查询时，table列是 <derivenN> 格式，表示当前查询依赖 id=N 的查询，于是先执行 id=N 的查询。

当有 union 时，UNION RESULT 的 table 列的值为<union1,2>，1和2表示参与 union 的 select 行 id。

NULL：mysql能够在优化阶段分解查询语句，在执行阶段用不着再访问表或索引。例如：在索引列中选取最小值，可以单独查找索引来完成，不需要在执行时访问表

```
mysql> explain select min(id) from film;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	(Null)	(Null)	(Null)	(Null)	(Null	(Null)	(Null)	(Null)	(Null)	Select tables optimized away

4. type列

这一列表示关联类型或访问类型，即MySQL决定如何查找表中的行，查找数据行记录的大概范围。

依次从最优到最差分别为：**system > const > eq_ref > ref > range > index > ALL**

一般来说，**得保证查询达到 range 级别，最好达到 ref**；

（1）const, system：mysql能对查询的某部分进行优化并将其转化成一个**常量**（可以看show warnings 的结果）。 system = 1。

用于 primary key 或 unique key 的所有列与常数比较时，所以表最多有一个匹配行，读取1次，速度比较快。system是const的特例，表里只有一条数据匹配时为 system；　（使用主键或者唯一索引的时候会出现）

```
mysql> set session optimizer_switch='derived_merge=off'; #关闭mysql5.7新特性对衍生表的合并优化
mysql> explain extended select * from (select * from film where id = 1) tmp;
mysql> set session optimizer_switch='derived_merge=on'; #还原默认配置
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	(Null)	system	(Null)	(Null)	(Null)	(Null)	1	100	(Null)
2	DERIVED	film	(Null)	const	PRIMARY	PRIMARY	4	const	1	100	(Null)

（2）eq_ref：primary key 或 unique key 索引的所有部分被连接使用 ，最多只会返回一条符合条件的记录。

这可能是在 const 之外最好的联接类型了，简单的 select 查询不会出现这种 type。

（在连接查询的时候，使用了主键或唯一索引的全部字段）

```
mysql> explain select * from film_actor left join film on film_actor.film_id = film.id;
```

```
mysql> explain select * from film_actor left join film on film_actor.film_id = film.id;
```

主键

联合索引中的一个字段

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	film_actor	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	100	(Null)
1	SIMPLE	film	(Null)	eq_ref	PRIMARY	PRIMA	4	tuling_	1	100	(Null)

说明： film_id：表的联合索引中的一个字段 ，但是 type 为 All;

因为使用的 * 查询的，指的要查询所有的字段，但是 film_actor 表的 remark 字段没有建立索引的，所以需要全表扫描；

（3）ref：相比 eq_ref，不使用唯一索引，而是使用普通索引 或者 唯一性索引的部分前缀，索引要和某个值相比较，可能会找到多个符合条件的行；

（使用的普通索引 或 联合唯一索引的部分前缀）

<1> 简单 select 查询，name是普通索引（非唯一索引）

```
mysql> explain select * from film where name = 'film1';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	film	ref	idx_name	idx_name	33	const	1	Using where; Using index

<2> 关联表查询，idx_film_actor_id 是 film_id 和 actor_id 的联合索引，这里使用到了 film_actor的左边前缀 film_id 部分。

```
mysql> explain select film_id from film left join film_actor on film.id = film_actor.film_id;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	film	index	(Null)	idx_name	33	(Null)	3	Using index
1	SIMPLE	film_actor	ref	idx_film_actor_id	idx_film_actor_id	4	test.film.id	1	Using index

（4）range：范围扫描通常出现在 in()，between ，> ， <，>= 等操作中。使用一个索引来检索给定范围的行。

```
mysql> explain select * from actor where id > 1;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	actor	range	PRIMARY	PRIMARY	4	(Null)	2	Using where

（5）index：扫描全表索引，通常比 All 快一些；

```
mysql> explain select * from film;
```

film 表的所有字段都建立了索引，使用 * 查询，则 type 为 index；

（6）ALL：即全表扫描，意味着mysql需要从头到尾去查找所需要的行。通常情况下这需要增加索引来进行优化了。

```
mysql> explain select * from actor;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	actor	ALL	(Null)	(Null)	(Null)	(Null)	3	(Null)

5. possible_keys 列

这一列显示查询 **可能** 使用哪些 **索引** 来查找。
explain 时可能出现 possible_keys 有值，而 key 显示 NULL 的情况，这是因为表中数据不多，mysql认为索引对此查询帮助不大，选择了全表查询。
如果该列是NULL，则没有相关的索引。在这种情况下，可以通过检查 where 子句看是否可以创造一个适当的索引来提高查询性能，然后用 explain 查看效果。

6. key 列

这一列显示mysql **实际** 采用哪个 **索引** 来优化对该表的访问。
如果没有使用索引，则该列是 NULL。如果想强制mysql使用或忽视 possible_keys 列中的索引，在查询中使用 force index、ignore index。

```
mysql> explain select * from film where name = 'film1';
```

信息 结果1 概况 状态

可能使用的索引 实际使用的索引

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	film	(Null)	ref	idx_name	idx_name	33	const	1	100	Using index

7. key_len列

这一列显示了**mysql 在索引里使用的字节数**，通过这个值可以算出具体使用了索引中的哪些列。
举例来说，film_actor的联合索引 idx_film_actor_id 由 film_id 和 actor_id 两个int列组成，并且每个int是4字节。通过结果中的key_len=4可推断出查询使用了第一个列：film_id列来执行索引查找。

```
mysql> explain select * from film_actor where film_id = 2;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	film_actor	(Null)	ref	idx_film_actor_id	idx_film_actor_id	4	const	1	100	(Null)

key_len计算规则如下：

```
字符串
    char(n): n字节长度
    varchar(n): 2字节存储字符串长度，如果是utf-8，则长度 3n + 2
数值类型
    tinyint: 1字节
    smallint: 2字节
    int: 4字节
    bigint: 8字节
时间类型
    date: 3字节
    timestamp: 4字节
    datetime: 8字节
```

注意：如果字段允许为 NULL，需要1字节记录是否为 NULL

索引最大长度是768字节，当字符串过长时，mysql会做一个类似左前缀索引的处理，将前半部分的字符提取出来做索引。

8. ref列

这一列显示了在 key 列记录的索引中，表查找值所用到的列或常量，常见的有：const（常量），字段名（例：film.id）。

9. rows列

这一列是mysql估计要读取并检测的行数，注意这个不是结果集里的行数。扫描的索引可能的行数

10. Extra列

这一列展示的是额外信息。常见的重要值如下：

Using index > Using index condition > Using where

（1）Using index：使用覆盖索引；覆盖索引是select的数据列只用从索引中就能够取得，不必读取数据行，换句话说查询列要被所建的索引覆盖。也就是查询的结果集中的所有字段都是在索引中的；

```
mysql> explain select film_id from film_actor where film_id = 1;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	film_actc	(Null)	ref	idx_film_actor_id	idx_film_actor_id	4	const	2	100	Using index

```
1 explain select film_id,remark from film_actor where film_id = 1;
```

不是索引

信息 结果1 概况 状态

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	film_actc	(Null)	ref	idx_film_actor_id	idx_film_actor_id	4	const	2	100	(Null)

（2）Using index condition：查询的列不完全被索引覆盖，where条件中是一个联合索引的前导列的范围；

```
mysql> explain select * from film_actor where film_id > 1;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	film_actor	range	idx_film_actor_id	idx_film_actor_id	4	(Null)	1	Using index condition

（3）Using where：使用 where 语句来处理结果，查询的列未被索引覆盖；

```
mysql> explain select * from actor where name = 'a';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	actor	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	33.33	Using where

（4）Using temporary：mysql需要创建一张临时表来处理查询。出现这种情况一般是要进行优化的，首先是想到用索引来优化。

<1> actor.name没有索引，此时创建了张临时表来 distinct；（distinct 查询可能会使用到临时表）

```
mysql> explain select distinct name from actor;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	actor	ALL	(Null)	(Null)	(Null)	(Null)	3	Using temporary

<2> film.name 建立了 idx_name 索引，此时查询时 extra 是 using index, 没有用临时表；将索引树加载到内存中，然后去重；

```
mysql> explain select distinct name from film;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	film	index	idx_name	idx_name	33	(Null)	3	Using index

（5）Using filesort：将用外部排序而不是索引排序，数据较小时从内存排序，否则需要在磁盘完成排序。这种情况下一般也是要考虑使用索引来优化的。

<1> actor.name未创建索引，会浏览actor整个表，保存排序关键字name和对应的id，然后排序name并检索行记录

```
mysql> explain select * from actor order by name;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	actor	ALL	(Null)	(Null)	(Null)	(Null)	3	Using filesort

<2> film.name建立了idx_name索引,此时查询时extra是using index

```
mysql> explain select * from film order by name;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	film	index	(Null)	idx_name	33	(Null)	3	Using index

（6）Select tables optimized away：使用某些聚合函数（比如 max、min）来访问存在索

引的某个字段；已经被 MySQL 优化过了；

```
mysql> explain select min(id) from film;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	Select tables optimized away

索引实践

示例表的创建

```
CREATE TABLE `employees` (
    `id` int(11) NOT NULL AUTO_INCREMENT,
    `name` varchar(24) NOT NULL DEFAULT '' COMMENT '姓名',
    `age` int(11) NOT NULL DEFAULT '0' COMMENT '年龄',
    `position` varchar(20) NOT NULL DEFAULT '' COMMENT '职位',
    `hire_time` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '入职时间',
    PRIMARY KEY (`id`),
    KEY `idx_name_age_position` (`name`,`age`,`position`) USING BTREE
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8 COMMENT='员工记录表';

INSERT INTO employees(name,age,position,hire_time) VALUES('LiLei',22,'manager',NOW());
INSERT INTO employees(name,age,position,hire_time) VALUES('HanMeimei',23,'dev',NOW());
INSERT INTO employees(name,age,position,hire_time) VALUES('Lucy',23,'dev',NOW());
```

1、全值匹配

```
EXPLAIN SELECT * FROM employees WHERE name= 'LiLei';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ref	idx_name_age_position	idx_name_age_position	74	const	1	Using index condition

name 是 联合索引 idx_name_age_position 的前导字段；

key_len 为 74，name为 varchar(24)，则 3 * 24 + 2 = 74，所以使用联合索引中的 name 字段走的索引；

```
EXPLAIN SELECT * FROM employees WHERE name= 'LiLei' AND age = 22;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ref	idx_name_age_position	idx_name_age_position	78	const,const	1	Using index condition

name，age 是联合索引 idx_name_age_position 的字段；

key_len 为 78，name为 varchar(24)，则 3 * 24 + 2 = 74；age 为 int ，所以值为4 ； 74 + 4 = 78，所以使用联合索引中的 name，age 字段走的索引；

```
EXPLAIN SELECT * FROM employees WHERE name= 'LiLei' AND age = 22 AND position ='manager';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ref	idx_name_age_position	idx_name_age_position	140	const,const,const	1	Using index condition

name，age，position 是联合索引 idx_name_age_position 的字段；

key_len 为 140，name为 varchar(24) 类型，则 3 * 24 + 2 = 74；age 为 int 类型，所以值为4；position 为 varchaer(20)，所以值为 3 * 20 + 2 = 62； 74 + 4 + 62 = 140，所以使用联合索引中的 name，age，position 字段走的索引；

2.最左前缀法则

如果建立了联合索引，要遵守最左前缀法则。指的是查询从联合索引的最左前列开始并且不跳过索引中的列。

（1）使用了联合索引的前两个字段查询；

```
EXPLAIN SELECT * FROM employees WHERE name = 'LiLei' and age = 22; #name, age 都走了索引
```

id	select_type	table	partit	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	(Null)	ref	idx_name_age_position	idx_name_age_position	78	const,const	1	100	(Null)

key_len 为 78，（3 * 24 + 2） + 4 = 78；走了 name 和 age 索引；

（2）使用联合索引的第1, 3 字段查询

```
EXPLAIN SELECT * FROM employees WHERE name = 'LiLei' and position ='manager'; #只有 name 走了索引
```

id	select_type	table	partiti	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	(Null)	ref	idx_name_age_po	idx_name_age_position	74	const	1	50	Using index condition

key_len 为 74，name 的 长刚好为 74，所以只有 name 走了索引；

分析：联合索引的底层存储是 先比较最前面的字段，最前面的字段一样则比较第2个字段，第2个一样才去比较第3个字段；第1个字段 name 去比较了，可以搜索到一部分， position 为 联合索引的第3个字段，但是索引在存储和查找时候不可能跳过第2个字段直接去比较第3个字段的，position 字段还是要列出大范围的数据做查询，因此 name 走了索引，position 没有走索引。

3. 不在索引列上做任何操作（计算、函数、（自动or手动）类型转换），会导致索引失效而转向全表扫描

（1）不给索引列进行函数操作

```
EXPLAIN SELECT * FROM employees WHERE left(name,3) = 'LiLei';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ALL	(Null)	(Null)	(Null)	(Null)	3	Using where

一般来说，只要给索引列增加了函数操作，MySQL的底层直接就不会使用索引去处理的。

（2）给 **hire_time** 增加一个普通索引：

```
ALTER TABLE `employees`
ADD INDEX `idx_hire_time` (`hire_time`) USING BTREE ;
```

```
EXPLAIN select * from employees where date(hire_time) ='2018-09-30'; #不会走索引
```

id	select_type	table	partitions	type	possible_key	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employee	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	3	100	Using where

针对以上的SQL，转化为日期范围查询，就会走索引：

```
EXPLAIN select * from employees where hire_time >='2018-09-30 00:00:00' and hire_time <='2018-09-3
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employee	(Null)	ALL	idx_time	(Null)	(Null)	(Null)	3	100	Using where

还原最初索引状态

```
ALTER TABLE `employees`
DROP INDEX `idx_hire_time`;
```

4.存储引擎不能使用索引中范围查找条件右边的列

联合索引的字段顺序，范围查找之后的列都不会去走索引；

```
EXPLAIN SELECT * FROM employees WHERE name= 'LiLei' AND age = 22 AND position ='manager'; #3个字段
EXPLAIN SELECT * FROM employees WHERE name= 'LiLei' AND age > 22 AND position ='manager';
```

id	select_type	table	partiti	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	(Null)	rang	idx_name_age_p	idx_name_age_position	78	(Null)	1	50	Using index condition

只会走前两个字段的索引。 第一个字段 name 使用的相等，所以可以找到具体的数据，第2个字段

会缩小到一个范围，第3个字段是在这个范围里做相等的查询，还是会要将这个范围去遍历一遍的，所以只有 name，age 走了索引。

5.尽量使用覆盖索引（只访问索引的查询（索引列包含查询列）），减少 select *语句

```
EXPLAIN SELECT name,age FROM employees WHERE name= 'LiLei' AND age = 23 AND position ='manager';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ref	idx_name_age_position	idx_name_age_position	140	const,c	1	Using where; Using index

```
EXPLAIN SELECT * FROM employees WHERE name= 'LiLei' AND age = 23 AND position ='manager';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ref	idx_name_age_position	idx_name_age_position	140	const,const,const	1	Using index condition

6. mysql在使用不等于（！=或者<>）的时候无法使用索引会导致全表扫描

```
EXPLAIN SELECT * FROM employees WHERE name != 'LiLei';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ALL	(Null)	(Null)	(Null)	(Null)	3	(Null)

7. is null, is not null 也无法使用索引

建议在建立字段的时候都设置为 not null，设置一个默认的值；

```
EXPLAIN SELECT * FROM employees WHERE name is null
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	(Null)	Impossible WHERE

8. like以通配符开头（'$abc...'）mysql索引失效会变成全表扫描操作

```
EXPLAIN SELECT * FROM employees WHERE name like '%Lei'
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ALL	(Null)	(Null)	(Null)	(Null)	3	Using where

```
EXPLAIN SELECT * FROM employees WHERE name like 'Lei%'
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	range	idx_name_age_position	idx_name_age_position	74	(Null)	1	Using index condition

模糊查找的时候，前模糊不走索引，后模糊会走索引；

因为在后模糊的时候，我们知道了这个字段的前面有几个字符，我们在索引中比较的只去比较前面的几个字符就好了；

问题：解决like '%字符串%'索引不被使用的方法?

（a）使用覆盖索引，查询字段必须是建立覆盖索引字段

```
EXPLAIN SELECT name,age,position FROM employees WHERE name like '%Lei%';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	index	(Null)	idx_name_age_position	140	(Null)	3	Using where; Using index

（b）如果不能使用覆盖索引则可能需要借助搜索引擎

9. 字符串不加单引号索引失效

```
EXPLAIN SELECT * FROM employees WHERE name = '1000'; #会走索引
EXPLAIN SELECT * FROM employees WHERE name = 1000;   #不会走索引
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	employees	ALL	idx_name_age_position	(Null)	(Null)	(Null)	3	Using where

因为 name 为字符串类型，MySql 会做隐式的类型转换，做了类型的转换，所以不会去走索引；

10.少用or或in，用它查询时，mysql不一定使用索引，mysql内部优化器会根据检索比例、表大小等多个因素整体评估是否使用索引，详见范围查询优化

```
EXPLAIN SELECT * FROM employees WHERE name = 'LiLei' or name = 'HanMeimei';
```

id	select_type	table	partiti	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	(Null)	ALL	idx_name_age_position	(Null)	(Null)	(Null)	2	100	Using where

11.范围查询优化

给年龄添加单值索引： ALTER TABLE `employees` ADD INDEX `idx_age` (`age`) USING BTREE；

```
explain select * from employees where age >=1 and age <=2000;
```

id	select_type	table	partiti	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	(Null)	ALL	idx_age	(Null)	(Null)	(Null)	2	100	Using where

从执行计划的结果可以看出，以上的范围查找不会走索引；

没走索引原因：mysql内部优化器会根据检索比例、表大小等多个因素整体评估是否使用索引。比如这个例子，可能是由于单次数据量查询过大导致优化器最终选择不走索引

优化方法：可以将大的范围拆分成多个小范围

```
explain select * from employees where age >=1 and age <=1000;
explain select * from employees where age >=1001 and age <=2000;
```

id	select_type	table	partiti	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	(Null)	rang	idx_age	idx_age	4	(Null)	1	100	Using index condition

还原最初索引状态：ALTER TABLE `employees` DROP INDEX `idx_age`;

索引使用总结（一般情况下）

建立了一个联合索引：（a， b，c）

Where语句	索引是否被使用
where a = 3	Y,使用到a
where a = 3 and b = 5	Y,使用到a，b
where a = 3 and b = 5 and c = 4	Y,使用到a,b,c
where b = 3 或者 where b = 3 and c = 4 或者 where c = 4	N
where a = 3 and c = 5	使用到a， 但是c不可以，b中间断了
where a = 3 and b > 4 and c = 5	使用到a和b， c不能用在范围之后，b断了
where a = 3 and b like 'kk%' and c = 4	Y,使用到a,b,c
where a = 3 and b like '%kk' and c = 4	Y,只用到a
where a = 3 and b like '%kk%' and c = 4	Y,只用到a
where a = 3 and b like 'k%kk%' and c = 4	Y,使用到a,b,c

like KK%相当于常量，所以走索引；%KK和%KK% 相当于范围，所以不走索引；

没走索引原因：mysql内部优化器会根据检索比例、表大小等多个因素整体评估是否使用索引。比如这个例子，可能是由于单次数据量查询过大导致优化器最终选择不走索引优化方法：可以讲大的范围拆分成多个小范围。